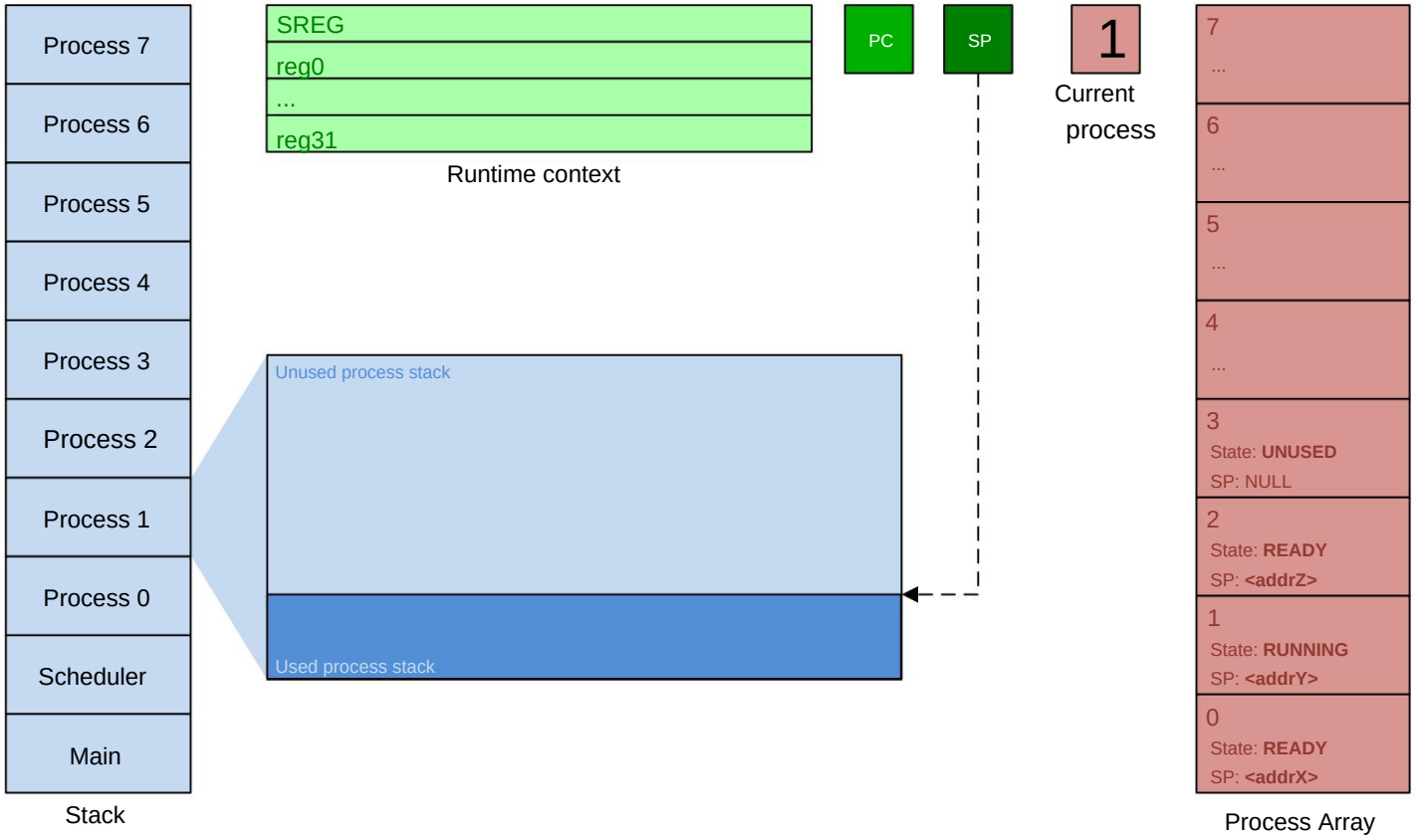
Machine Translated by Google

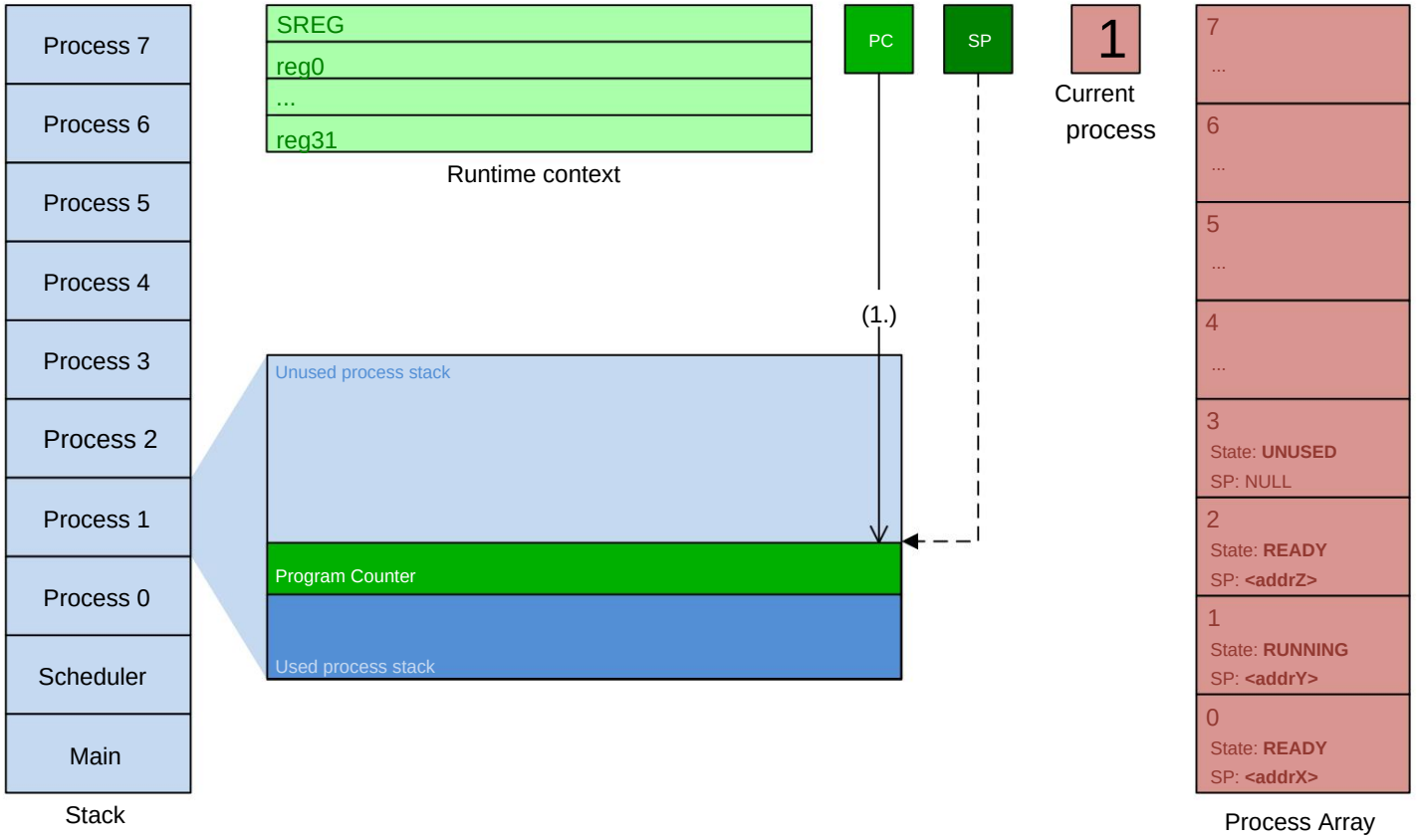
Initial state: Process track of process 1



The figure above illustrates the state of SPOS before the process change, with the stack on the right, the microcontroller registers at the top, and data structures for process management on the left.

Machine Translated by Google

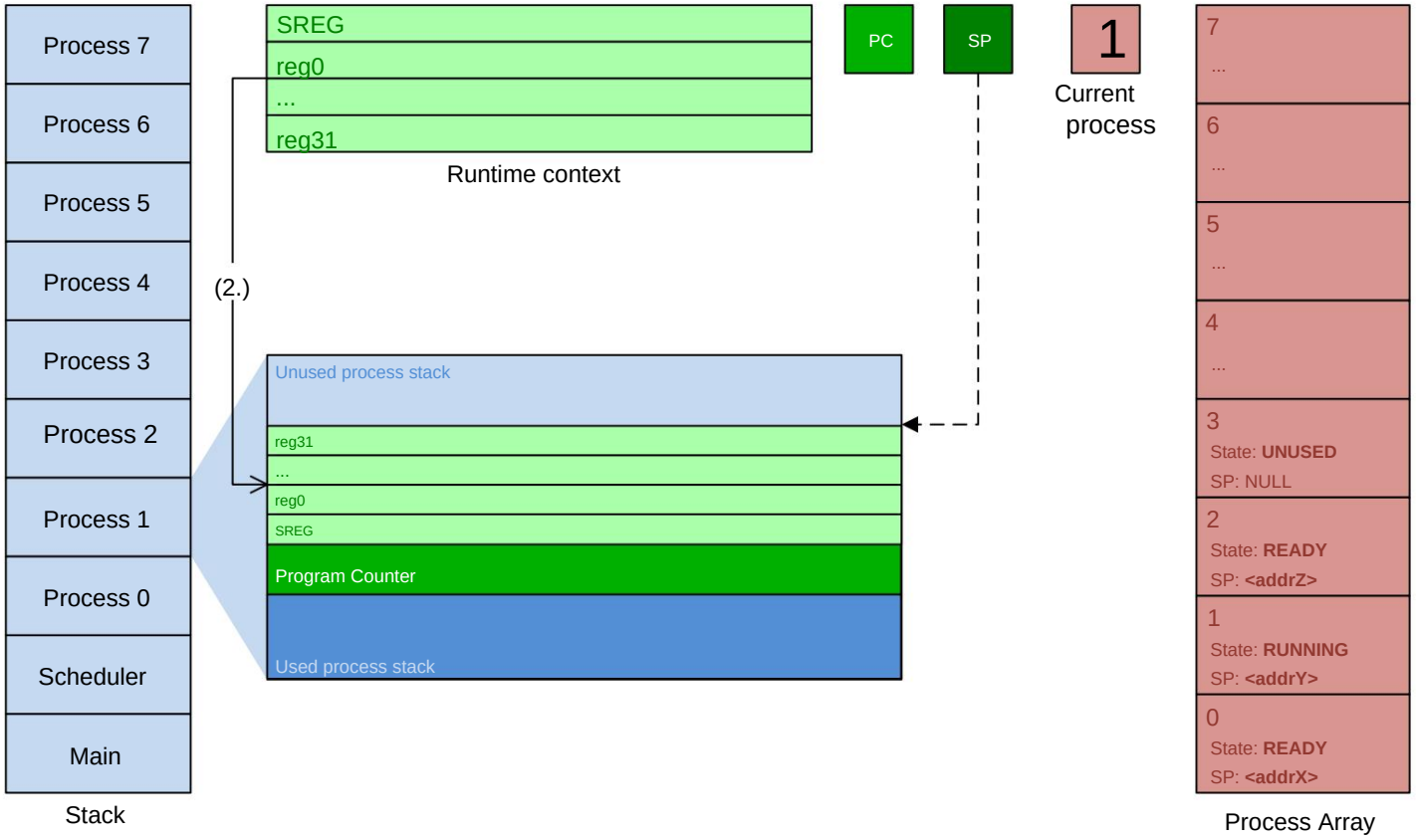
Step 1: Saving the program counter



When the process jumps to an ISR, the program counter is implicitly stored as the return address for the later continuation of the current process on its process stack. The stack pointer then points to the first free position on the process stack.

Machine Translated by Google

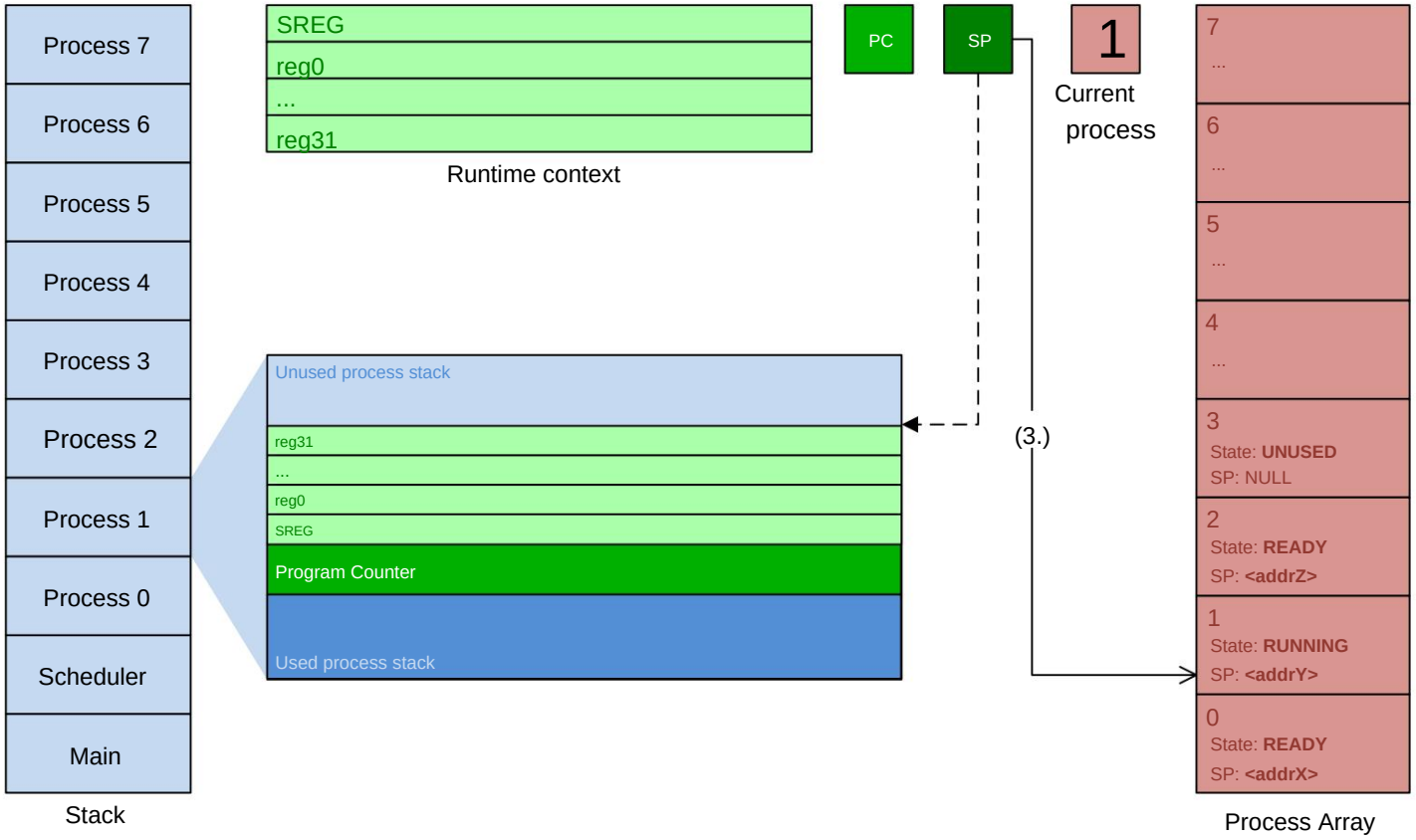
Step 2: Saving the runtime context



Calling the assembler macro ` **saveContext()**` saves the runtime context to the scheduler's process stack at the time of interruption. The stack pointer then points to the first available slot in the process stack.

Machine Translated by Google

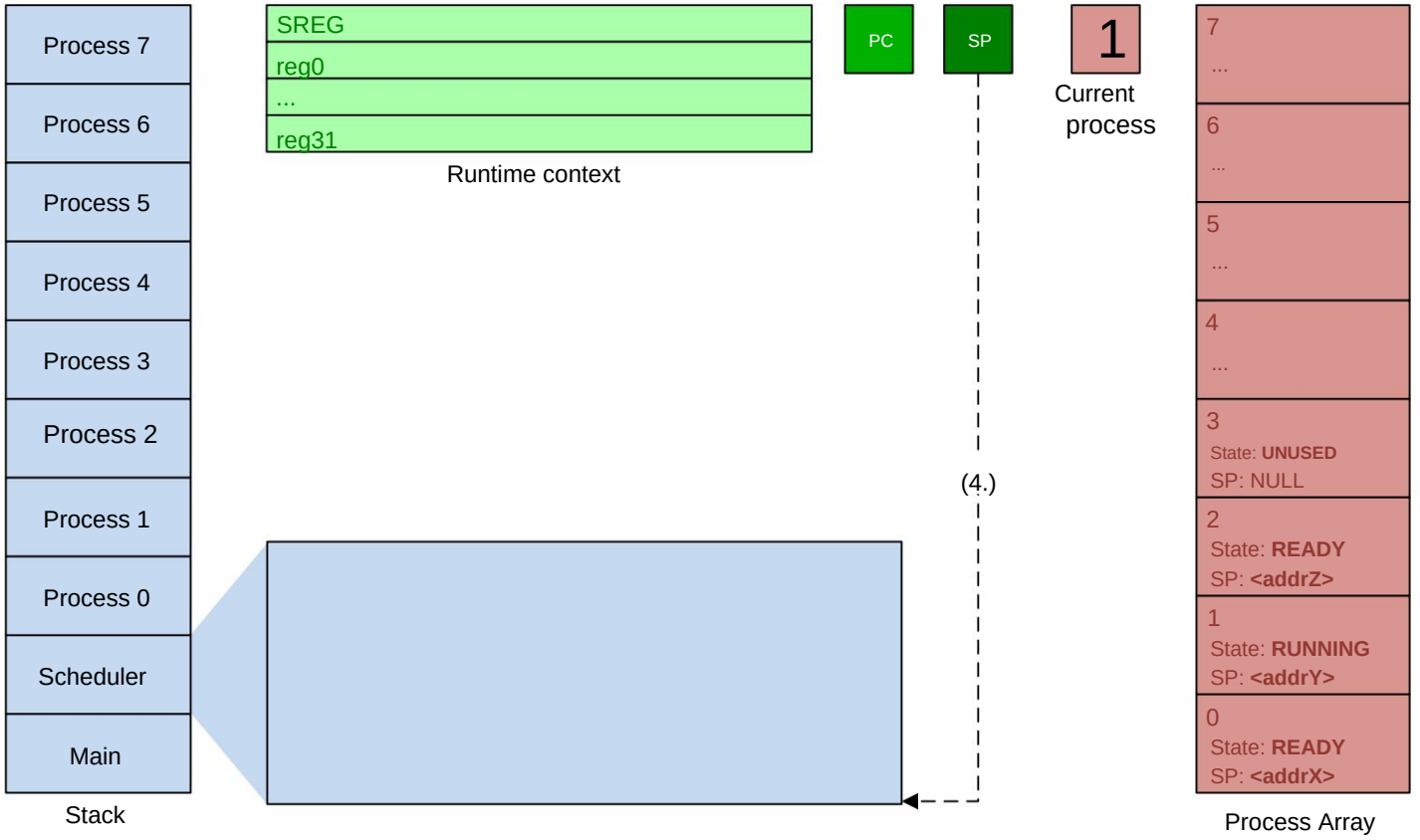
Step 3: Saving the stack pointer



To later restore the runtime context for process 1, the current address of the stack pointer is saved in the process array.

Machine Translated by Google

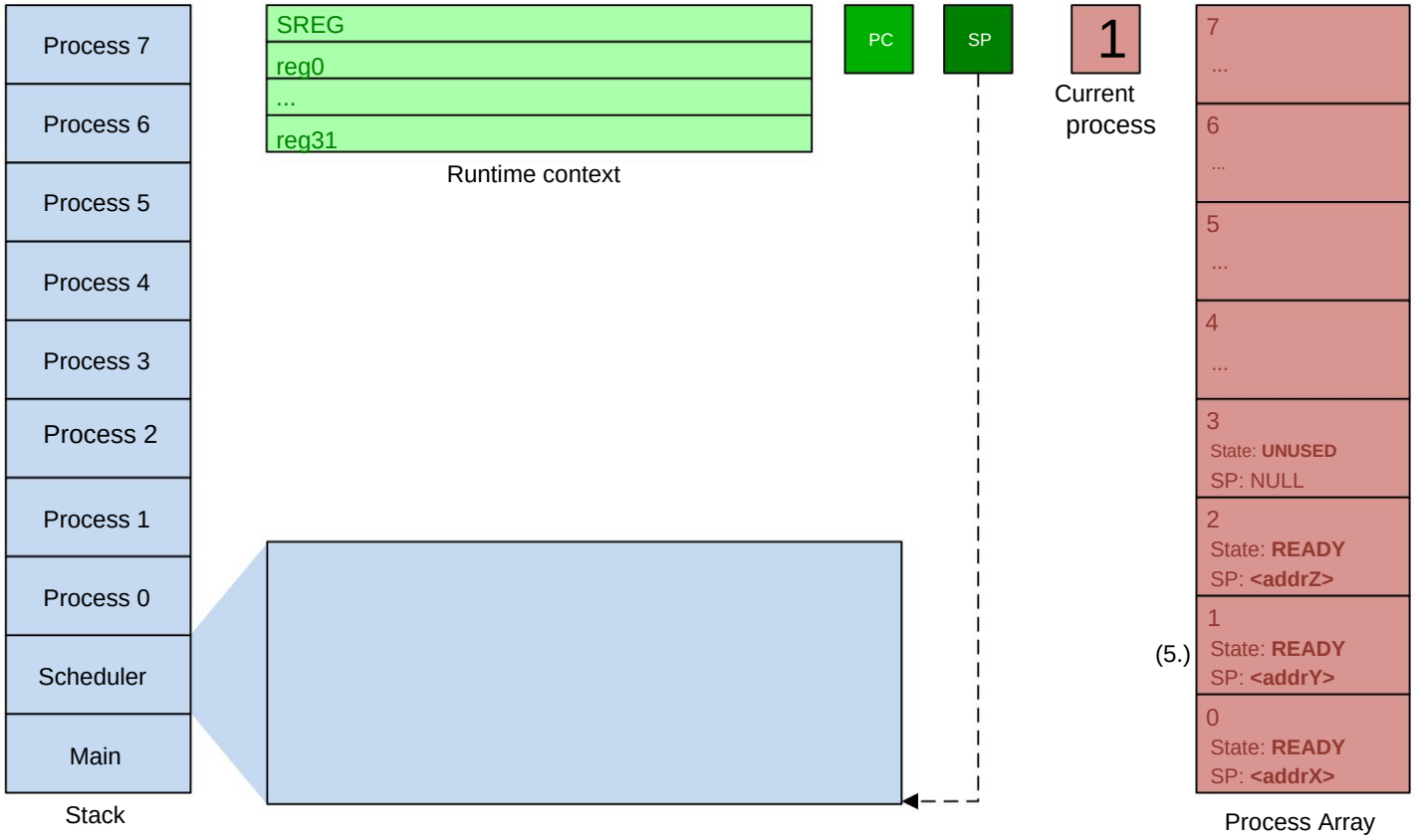
Step 4: Setting the stack pointer to the ISR stack



For the actual execution of the scheduler ISR, the stack pointer is set to the beginning of the ISR stack.

Machine Translated by Google

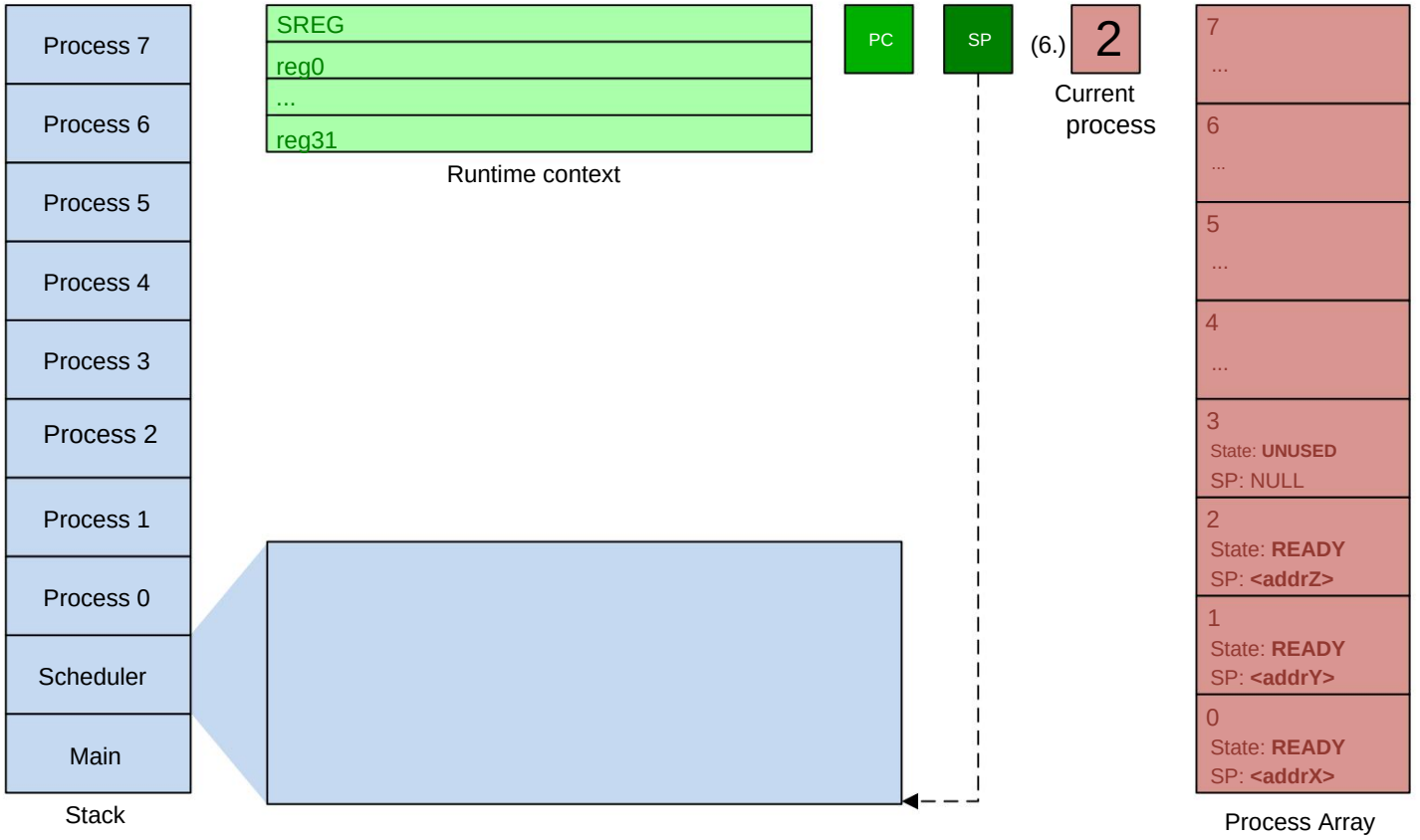
Step 5: Current process set to OS_PS_READY



The process state of the current process is set to **OS_PS_READY** in the process array .

Machine Translated by Google

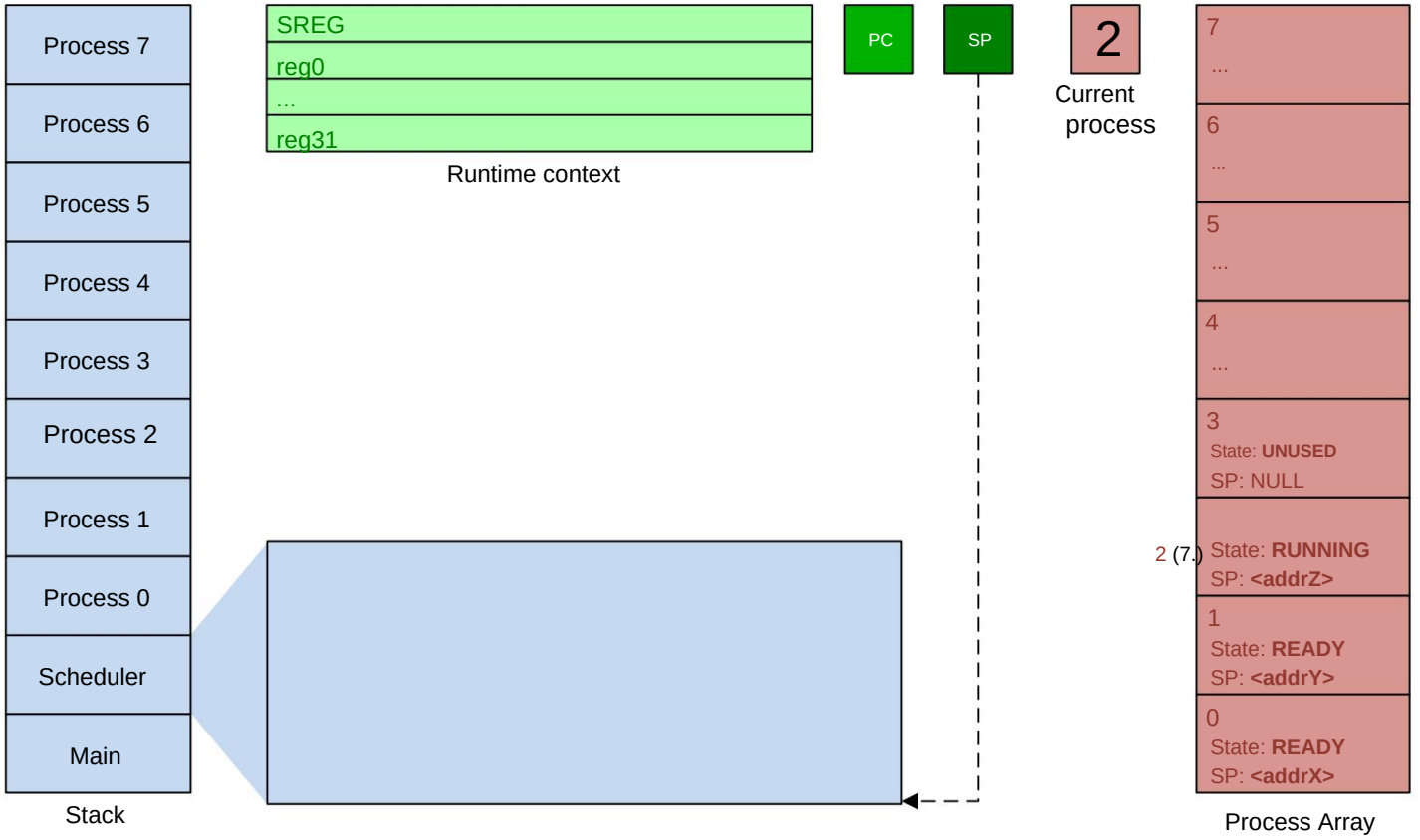
Step 6: Selection of the next process



Using the currently used scheduling strategy, the next process to be continued is selected, which in our example is process 2.

Machine Translated by Google

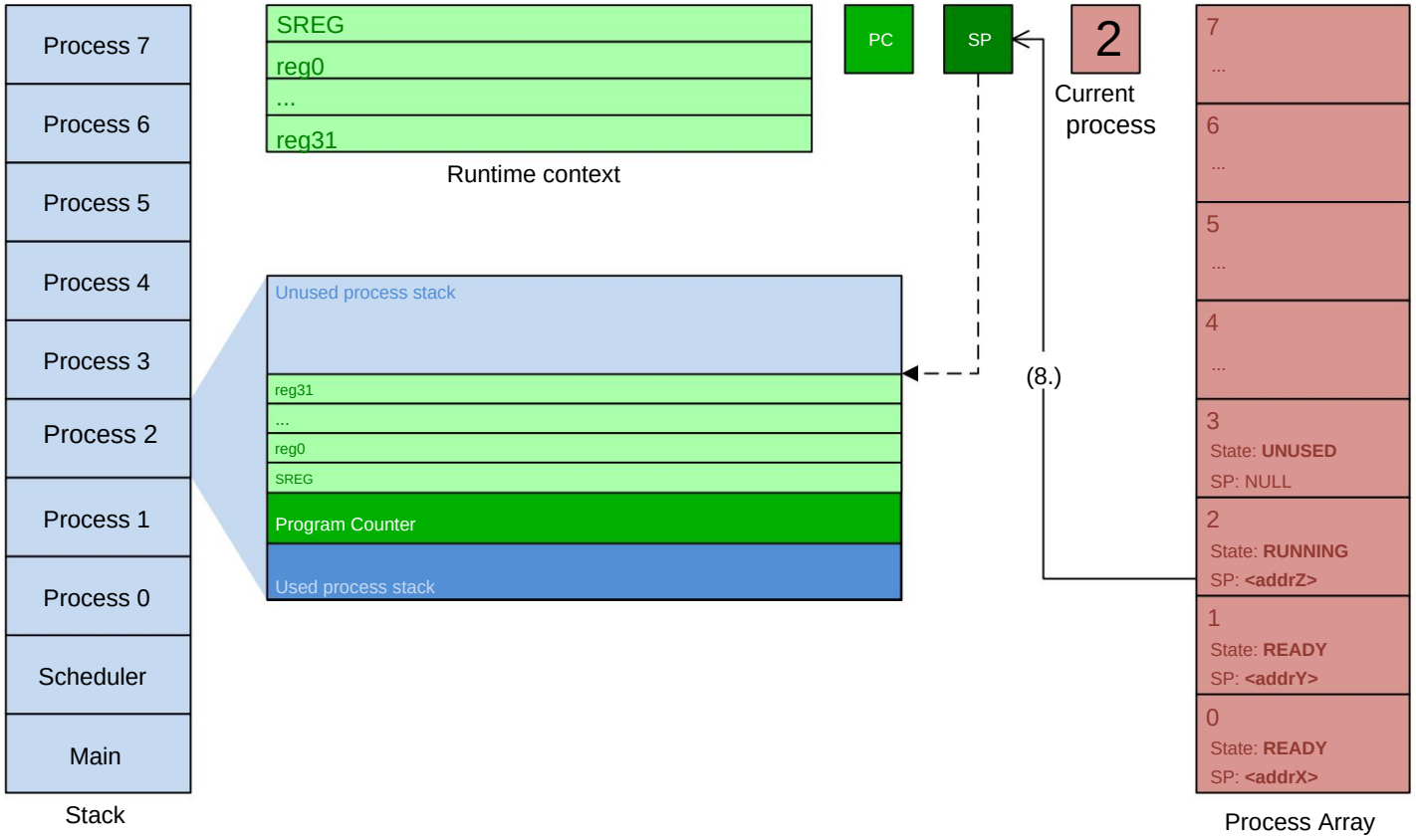
Step 7: Next process on OS_PS_RUNNING



After process 2 has been selected as the next process to continue, its process state in the process array is set to **OS_PS_RUNNING .**

Machine Translated by Google

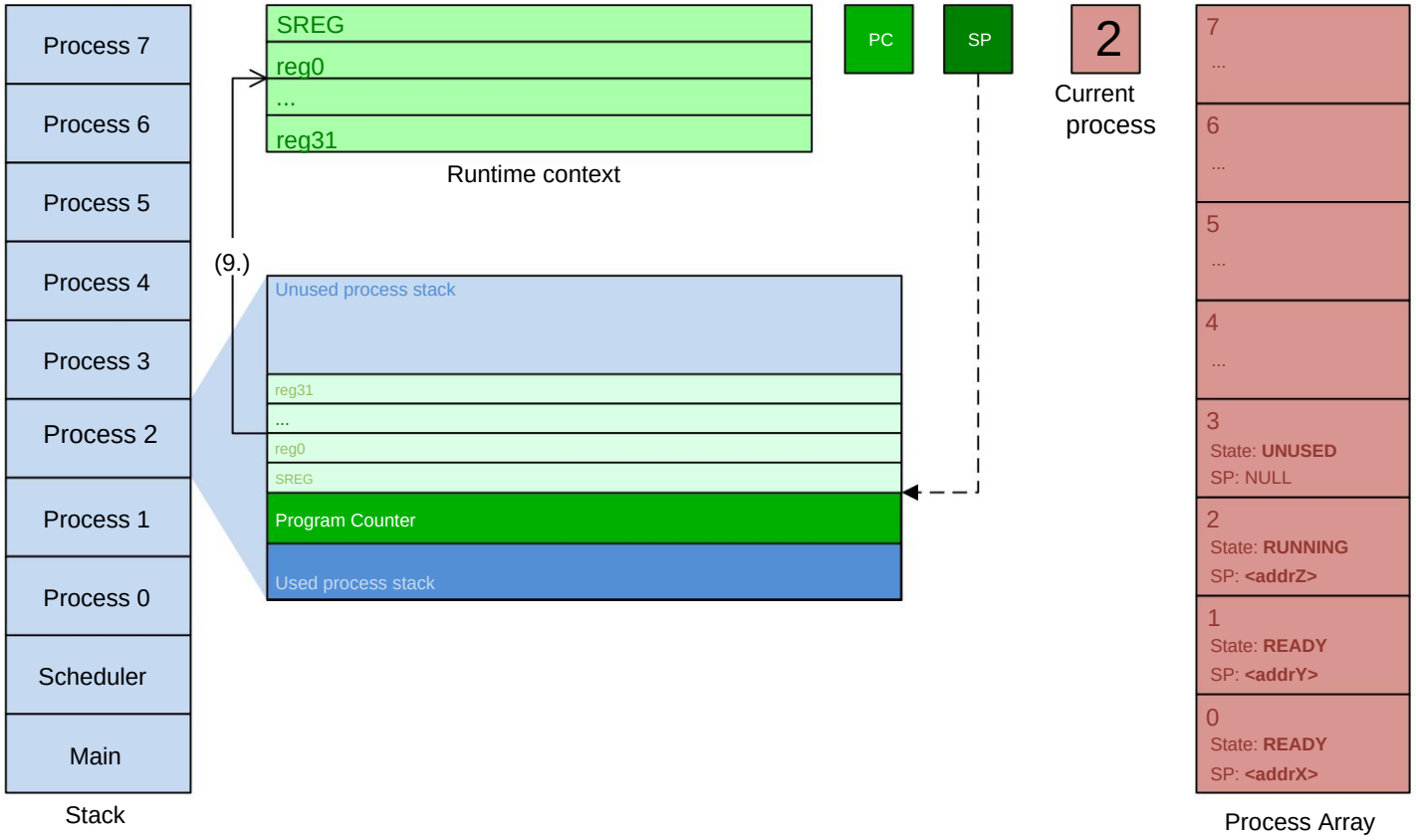
Step 8: Restore the Stack Pointer



The stack pointer address, previously saved in the process array and pointing to the first free position above the runtime context, program counters, and the remaining used process track of process 2, is restored in this step. This step is equivalent to step 3, "*Saving the Stack Pointer .*"

Machine Translated by Google

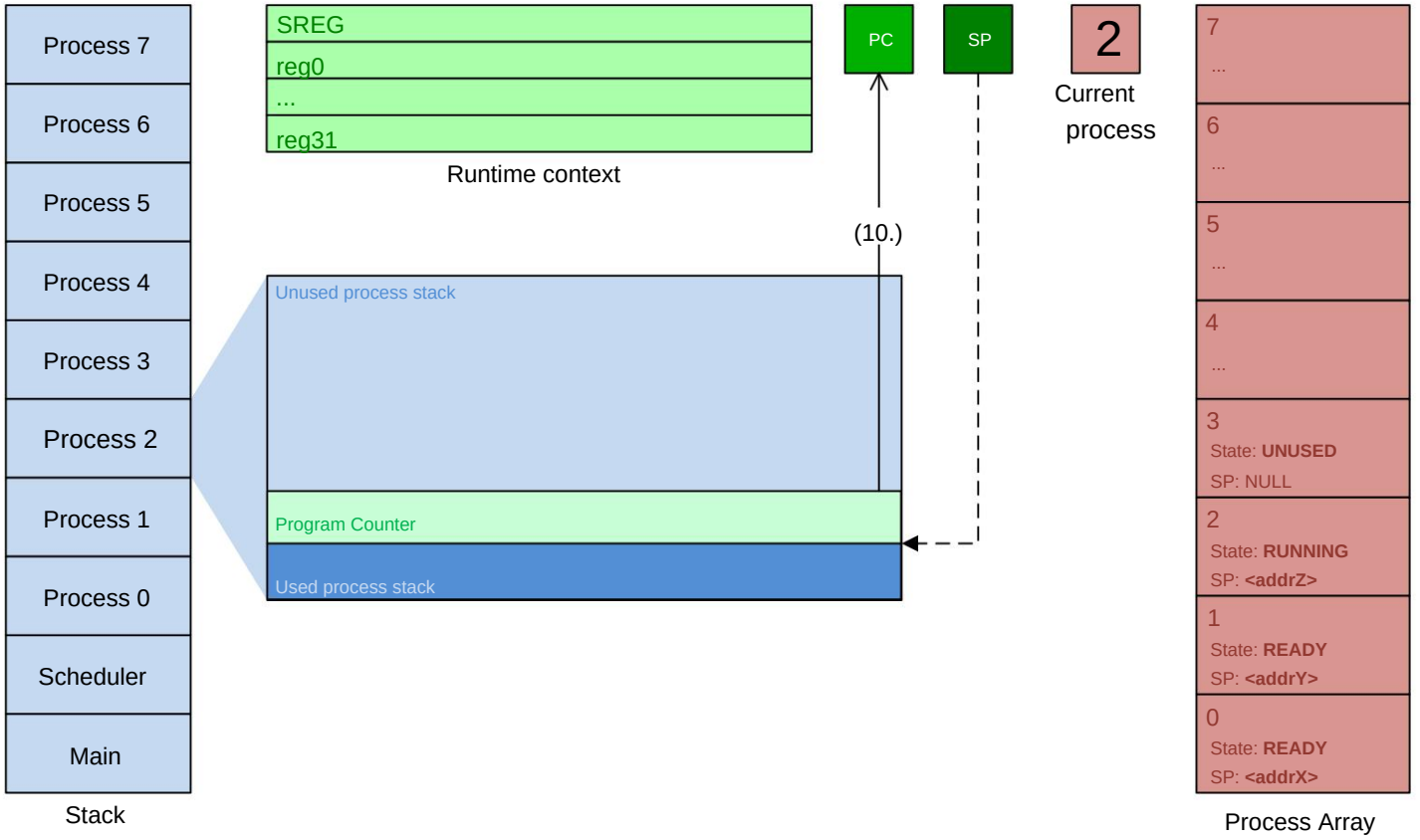
Step 9: Restore the runtime context



The assembler macro ` **restoreContext()** ` restores the runtime context of Process 2 that was saved at an earlier time. This step is equivalent to step 2, " Saving *the Runtime Context" .*

Machine Translated by Google

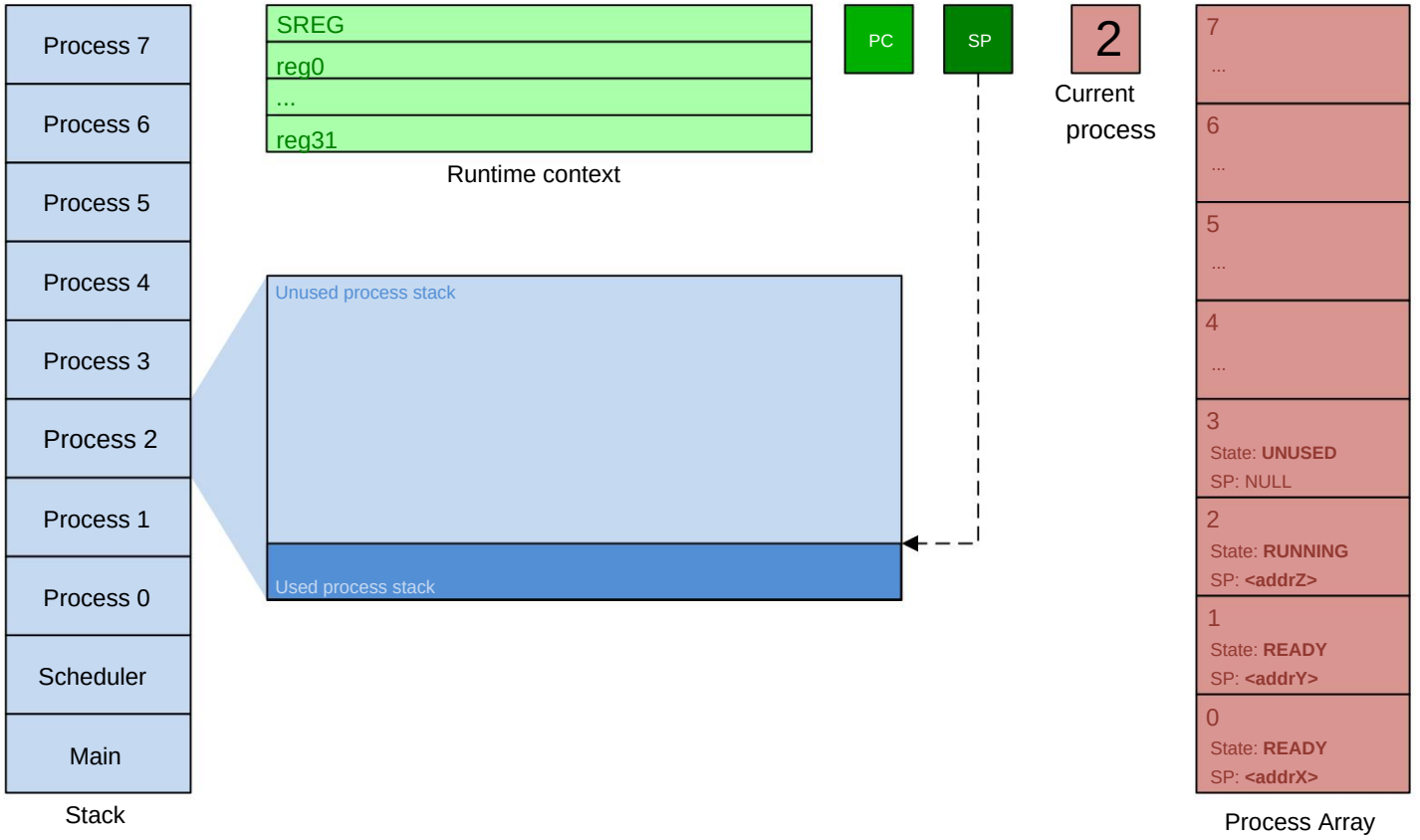
Step 10: Return



The **`reti`** instruction contained in the assembler macro ` **restoreContext()`** also automatically returns the program to the point in the flow where the process was previously interrupted by the scheduler. This step is equivalent to step 1, *saving the program counter .*

Machine Translated by Google

Final state: Process stack of process 2



The resumed process now has access to the runtime context it had before the scheduler interrupted it. Execution continues from the point where it was previously interrupted by the scheduler, as the registers and program counter previously placed on the stack have been restored.